

מספר קורס: 0368.2158

25.3.2024, ט"ו אדר ב

סמסטר א', מועד א', תשפ"ד

פתרון מבחן במבני נתונים

ד"ר רני הוד, פרופ' יוסי עזר, נתן גייר, דני דורפמן, שחר לבקוביץ

משך המבחן שלוש שעות

הבחינה ללא מחשבים/מחשבוניס. מותר להביא דף דו נוסחאות דו צדדי.

הוראות כלליות:

- יש לרשום מספר ת.ז. ומספר מחברת בראש כל דף.
- מותר להביא דף נוסחאות דו צדדי. ניתן להביא דף דו צדדי נוסף לקבוצה 28.
- המחברות הן לטייטה בלבד ולא תיבדקנה.
- מותר להשתמש באלגוריתמים שנלמדו בכיתה כקופסאות שחורות.
- אין לכתוב אלגוריתמים בפסאודו-קוד אלא להסביר (באופן משכנע) את הרעיון הכללי.
- כתבו תשובות קצרות ומדויקות. תשובות לא ממוקדות, ארוכות, מסובכות או מסורבלות יקבלו ניקוד חלקי גם אם הן נכונות. עם זאת, תשובה ללא נימוק מדויק לא תזכה בנקודות.
- סיבוכיות זמני ריצה יש לכתוב כחסם הדוק ככל שתוכלו, בכתב $O()$. יש לנתח ולנמק את הסיבוכיות.
- אלא אם נאמר אחרת, על האלגוריתמים להיות דטרמיניסטיים (אלא אם כן כתוב בתוחלת) וזמני הריצה הם worst-case (אלא אם כן כתוב אמורטייזד).
- יש לענות על כל שאלה במסגרת המקום המוקצה, בלי חריגות פרט לשימוש ב"דף החירום" הנוסף.
- כשמבקשים חציון של מספר איברים זוגי, הכוונה לחציון תחתון.
- בכל השאלות ניתן להניח שהמפתחות ייחודיים, אלא אם נאמר אחרת.

בהצלחה!!!

א. [REDACTED]

ב. [REDACTED]

סעיף א

מבנה הנתונים: מערך מעגלי שנעשה לו doubling. נשמור שדה גלובאלי M שמציין את גודל המערך הנוכחי, שדה גלובאלי n שמציין את מספר האיברים במבנה ושדה גלובאלי $last$ שיצביע לסוף התור.

מימוש הפעולות:

- $enqueue(x)$ – ראשית, במידה ו $n = M$, נעתיק את האיברים למערך בגודל $2M$ החל מאינדקס 1 ונשנה את $last$ ל $M + 1$. שנית, נכניס את x למקום $last$ ולאחר מכן נגדיל ב1 את n ואת $last$.
- $dequeue()$ – האיבר שיש להחזיר נמצא באינדקס $(last - n) \% M$. נקטין את n ב-1. אם $n = M/4$ אז נעתיק את האיברים למערך בגודל $M/2$ החל מאינדקס 0 ונשנה את $last$ ל $M/4 + 1$.
- $Retrieve(i)$ – נחזיר את האיבר במקום $(last - n + i - 1) \% M$.

מקום: מתקיים $M/4 \leq n \leq M$ ולכן כמות המקום שמבנה הנתונים צורך היא לינארית במספר האיברים ששמורים במבנה.

ניתוח סיבוכיות: כל הפעולות מתבצעות בזמן אמורטייזד $O(1)$. ניתן להוכיח באמצעות פונקציית הפוטנציאל

$$\phi(M, n) = \begin{cases} 2n - M, & n \geq M/2 \\ M/2 - n, & n \leq M/2 \end{cases}$$

סעיף ב

מבנה הנתונים: נבצע דה אמורטיזציה. נשמור בכל רגע נתון 3 מערכים מעגליים בגדלים $M, 2M, \frac{M}{2}$ כאשר תמיד המערך הפעיל שמכיל את התור יהיה המערך בגודל M .

נממש את הפעולות כמו במימוש הקודם עם השינוי הבא:

- אם $n > M/2$ אז בכל פעולה נעתיק 1000 איברים מהמערך באורך M למערך באורך $2M$ (פעם ראשונה נעתיק את 1000 האיברים הראשונים בתור, אחכ את 1000 הבאים וכולי).
- אם $n < M/2$ אז בכל פעולה נעתיק 1000 איברים מהמערך באורך M למערך באורך $M/2$ (פעם ראשונה נעתיק את 1000 האיברים הראשונים בתור, אחכ את 1000 הבאים וכולי).
- כשמוחקים איבר ב $dequeue$, יש למחוק אותו מכל המערכים בהם הוא מופיע.
- אם n משנה גודל מיותר מ $M/2$ לפחות מ $M/2$ או להיפך, אז אנו דורסים את עבודת העתקה שביצענו עד כה ומעתלמים ממנה.

אם $n = M$ אז את המערך הקטן בגודל $\frac{M}{2}$ זורקים. המערך הבינוני הופך לקטן, המערך הגדול (שכבר מכיל את כל האיברים) הופך לבינוני ומקצים מערך בגודל $4M$ שישמש כמערך הגדול. טיפול דומה קורה במקרה שבו $n = M/4$.

כל הפעולות רצות בזמן ריצה $O(1)$ במקרה הגרוע וגודל המקום שהמבנה צורך הוא $O(n)$.

שאלה 2 (20 נקודות)

א.

ב.

הטריק הקלאסי בשאלות מסוג זה הוא לבודד משתנים:

$$j - i = |a_j - a_i| \text{ אם } j - i = a_j - a_i \text{ או } j - i = a_i - a_j \text{ אם } j - i = a_j - a_i \\ a_i - i = a_j - j \quad \vee \quad a_i + i = a_j + j$$

סעיף א

אלגוריתם – נעזר ב-2 מילונים במימוש *hash table* בגודל n בשיטת *chaining*. נסמן את המילונים ב- D_1, D_2 .

עבור $i = 1 \dots n$:

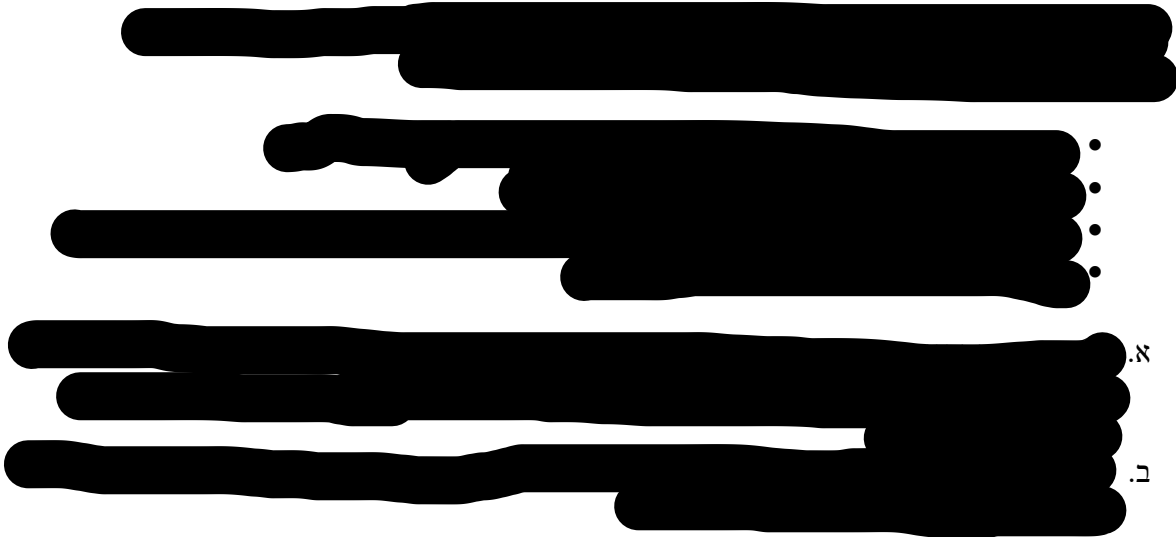
- נבדוק האם $a_i - i$ מופיע ב- D_1 . אם כן נחזיר *true*. אחרת, נכניס את $a_i - i$ ל- D_1 .
 - נבדוק האם $a_i + i$ מופיע ב- D_2 . אם כן נחזיר *true*. אחרת, נכניס את $a_i + i$ ל- D_2 .
- במידה ולא החזרנו *true* בלולאה, נחזיר *false*.

סיבוכיות $O(n)$ בתוחלת (כמות לינארית של פעולות על טבלת *hash* בגודל n).סעיף ב

ניצור 2 מערכים, אחד עם הערכים $a_i + i$, השני עם $a_i - i$. נבחין כי כל הערכים שלמים בתחום $\{-n^2 - n, \dots, n^2 + n\}$ ולכן אנו יודעים למיין אותם בזמן לינארי עם *radix sort*. כל שנותר הוא לבדוק האם יש זוג איברים זהים באחד מהמערכים, ומספיק להסתכל על זוגות עוקבים בזמן לינארי כיוון שהם ממוינים.

סיבוכיות $O(n)$.

שאלה 3 (20 נקודות)



סעיף א

מבנה הנתונים – 2 עצי AVL. אחד עבור f המפתחות הקטנים בעץ ואחד עבור $n - f$ המפתחות הגדולים. בהתחלה (לפני שאילתת focus) יהיה לנו עץ AVL בודד שהוא ריק. העצים ישמרו שדה $size$ בשביל שאילות select. בנוסף העץ עם האיברים הקטנים ישמור מצביע למקסימום והעץ השני ישמור מצביע למינימום. בנוסף יהיה לנו משתנה גלובלית f שישמור את ה"focus".

- $Insert(key, val)$ – במידה ו f עוד לא אותחל אז נכניס רגיל לעץ AVL הבודד של מבנה הנתונים. אחרת. נשווה את key למקסימום של f האיברים הקטנים ולפי זה נדע לאיזה עץ להכניס את key . במידה והכנסנו את key לעץ של המפתחות הקטנים אז נמחק שם את המקסימום ונעבירו לעץ של המפתחות הגדולים.
- $delete(key)$ – במידה ו f עוד לא אותחל אז נמחק רגיל מעץ AVL הבודד של מבנה הנתונים. אחרת. נשווה את key למקסימום של f האיברים הקטנים ולפי זה נדע מאיזה עץ למחוק את key . במידה ומחקנו את key מהעץ של המפתחות הקטנים אז נמחק את המינימום בעץ של המפתחות הגדולים ונעבירו לעץ של המפתחות הקטנים.
- $setFocus(f)$ – נמזג את עצי ה AVL ונעשה להם $split$ באיבר מדרגה f . נעדכן את משתנה ה $focus$ הגלובאלי שלנו ל f .
- $select(i)$ – נבצע $select$ באמצעות המצביע למינימום/מקסימום של העץ המתאים.

סיבוכיות – נובע מסיבוכיות AVL finger tree.

סעיף ב

אי אפשר. נניח בשלילה שקיים מבנה נתונים כזה ונציע אלגוריתם למיון מערך באורך n :

1. נכניס את איברי המערך למבנה הנתונים.

2. עבור $i = 1 \dots n$: נעשה

$SetFocus(i)$ a.

$Select(i)$ b. (ונדפיס את האיבר הזה)

עלות שלב 1 היא $O(n \log n)$. סה"כ עלות שלב a היא $O(n \log n)$. סה"כ עלות שלב b היא $\Theta(n)$. כלומר הצלחנו למיין ב $O(n \log n)$, בסתירה.

שאלה 4 (20 נקודות)סעיף א

מבנה הנתונים – ערימה בינארית H ומחסנית S (הממושת ע"י רשימה מקושרת דו-כיוונית). כל מפתח מוכנס לשתייהן ושומרים מצביעים דו כיווניים זה לזה.

- $push(key)$ – נבצע $H.insert(key), S.push(key)$ ונקשר ביניהם.
- $pop()$ – נבצע $S.top()$, בעזרתו נקבל מצביע למיקום בערימה ואז נמחק מהמחסנית ומהערימה.
- $min()$ – מחזיר את המפתח המינימלי ב- H .
- $deleteMin()$ – נבצע $H.min()$ ובעזרתו נקבל מצביע לצומת במחסנית. כעת נמחק גם מהמחסנית וגם מהערימה.

כל הפעולות על המחסנית הן בזמן קבוע (על ראש הרשימה או דרך מצביע לאיבר) ולכן הזמנים מוכתבים ע"י הפעולות על הערימה, שעומדות בדרישות הסיבוכיות.

סעיף ב

נחליף את הערימה בערימה בינומית עצלה (או פיבונאצ'י). מימוש הפעולות נותר זהה, וכיוון ששוב זמני הפעולות על הערימה הם הדומיננטיים, המימוש עומד בדרישות הסיבוכיות אמורטיזד.

הערה: היו גם פתרונות נכונים נוספים שכללו הכנסת המפתחות במקביל לשני מבני נתונים, אחד עבור סדר המפתחות ואחד עבור סדר ההכנסה (דוגמאות: ערימת מינימום וערימת מקסימום, עץ AVL ורשימה מקושרת דו-כיוונית).

שאלה 5 (20 נקודות)

[Redacted]

א. [Redacted]

ב. [Redacted]

[Redacted]

תוודאו שאתם מבינים למה בסעיף א לוקחים חציון על פני $\frac{1}{3} \cdot \frac{3}{8}n + \frac{1}{5} \cdot \frac{5}{8}n = \frac{n}{4}$ איברים
ובסעיף ב לוקחים חציון על פני $0.1n + 0.14n = 0.24n$ איברים.

סעיף א

במקרה הגרוע החציון שייבחר גדול מ $\frac{1}{8}n$ חציוני השלשות וקטן מ $\frac{1}{8}n$ חציוני החמישיות (או
להפך). במקרה זה רק נוכל להבטיח ש $2 \cdot \frac{n}{8} = \frac{n}{4}$ מפתחות יזרקו והאלג ימשיך ברקורסיה עם
מערך מגודל לכל היותר $\frac{3}{4}n$.

מקבלים נוסחת נסיגה של $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3}{4}n\right) + \Theta(n)$. כלומר $\Theta(n \log n)$.

סעיף ב

הפעם הסיבוכיות כן תהיה לינארית כי שיפרנו קצת את היחס.

במקרה הגרוע החציון שייבחר גדול מ $0.1n$ חציוני השלשות ומ $0.24n/2 - 0.1n = 0.02n$ חציוני
החמישיות. באותו אופן החציון גם יהיה קטן גם מ $0.1n$ חציוני השלישיות ו $0.02n$ מחציוני
החמישיות. לכן יזרקו לפחות $2 * 0.1n + 3 * 0.02n = 0.26n$ מפתחות. כלומר
האלגוריתם ימשיך רקורסיבית עם מערך בגודל לכל היותר $0.74n$.
מקבלים נוסחת נסיגה של $T(n) = T(0.24n) + T(0.74n) + \Theta(n)$. כלומר $\Theta(n)$.

טעויות נפוצות

שאלה 1

היו סטודנטים שתיארו מבנה נתונים שדומה למחסנית במקום לתור ושלא השתמשו במערך מעגלי. יש צורך להשתמש במערך מעגלי על מנת ש-retrieve ירוץ בזמן קבוע במקרה הגרוע - עצי AVL ו-hash לא מתאימים כאן. צריך לבצע הקטנה של המערך פי 2 כאשר מספר האיברים הוא בגודל $M/4$ (ולא $M/2$) על מנת שבכל רגע, גודל הזיכרון שמשמש את מבנה הנתונים יהיה לינארי במספר האיברים שנמצאים בו. היו סטודנטים שלא פירטו מספיק: לא הסבירו מה המבנה כולל, בפרט מה קורה במקרי הקצה (מערך מתמלא/מתרוקן). היה צורך להסביר את ניתוח האמורטייזד על ידי הצגת פונקציית הפוטנציאל או הסבר על השימוש במטבעות.

שאלה 2

הרבה סטודנטים ניסו להכניס מפתחות מהצורה $a_i + i$ יחד עם $a_i - i$ לאותו מילון, אך במצב זה יכולה להיות גם התנגשות לא חוקית של $a_i - i = a_j + j$. היו סטודנטים שניסו להתמודד עם הבעיה ע"י כך שישמרו במידע של המפתח ביט שמציין את סוג הפעולה, אך לא שמו לב שביט אחד אינו מספיק כי קיים מקרה שבו רוצים להכניס גם את הפעולה השנייה. באופן דומה, סטודנטים ששמרו במידע את האינדקס ובדקו את קיום התנאי מולו שגו מאותה סיבה, שלא התבצעה הכנסה כאשר הבדיקה נכשלת.

שאלה 3

- א. סעיף א - סטודנטים רבים השתמשו בעץ AVL בודד עם מצביע לאיבר f בגודלו ואמרו שאפשר לעשות כמו בכיתה selection החל מה finger הזה. הפתרון לא מספיק יעיל. דוגמה פשוטה היא במידה זה finger כעת מצביע לשורש ומתבצעת שאילתת $select(f + 1)$. הסיבוכיות המבוקשת לשאילתה הזאת היא קבועה אך העוקב של השורש נמצא במרחק לוגריתמי ממנו.
- ב. סעיף א – בהמשך לפתרון הקודם. סטודנטים מימושו את $select(i)$ על ידי ביצוע פעולות successor/predecessor מה finger. אין סיבה שזה יהיה מספיק יעיל.
- ג. סעיף ב – בלא מעט פתרונות בהוכחת החסם התחתון סטודנטים הניחו שהאלגוריתם הכללי (שלו הם מראים חסם תחתון) הוא מבוסס עץ AVL ונעזרו בסריקת *in order*. אין סיבה שכל אלגוריתם לפתרון הבעיה ישתמש בעצי AVL.

שאלה 4

שמירת מצביע לאיבר האחרון שהוכנס בלבד (אי אפשר לעדכן אותו במהירות אחרי pop); שימוש ברשימה מקושרת חד-כיוונית (אי אפשר למחוק מהאמצע בזמן קבוע).

שאלה 5

בשני הסעיפים חלק מהסטודנטים לא חישבו נכון את מספר הקבוצות שעליהם צריך לקחת חציון. בנוסף חלק מהסטודנטים לא שמו לב שיתכן שבמקרה הגרוע ביותר כל השלישיות בצד אחד וכל החמישיות בצד השני. חלק מהסטודנטים בצעו רקורסיה נפרדת לחמישיות והשלישיות שזה כמובן לא מתאים.